

Football ETL Pipeline

Documentação Técnica Completa

Pipeline de dados em Python (ETL) com histórico em SQLite, relatório em PDF e execução automatizada

Vitor Miranda Jeremias

Estudante de Engenharia de Software - PUCPR
github.com/Vitormjere

Sumário

- 1. Visão Geral
- 2. Arquitetura Geral
- 3. Extract - Coleta de Dados
- 4. Transform - Processamento e Regras
- 5. Load - Persistência do Histórico
- 6. Report - Geração do PDF
- 7. Automação - Execução Agendada
- 8. Testes Automatizados
- 9. Desafios Reais e Como Foram Resolvidos
- 10. Como Rodar o Projeto Localmente

1. Visão Geral

O Football ETL Pipeline é um projeto desenvolvido em Python que automatiza a coleta e o processamento de dados de futebol. Ele extrai resultados e classificações de competições como Premier League, Champions League e La Liga por meio de uma API pública, calcula métricas de desempenho, mantém um histórico dessas informações e gera automaticamente um relatório em PDF.

Diferente dos meus projetos anteriores em C#/.NET, que eram focados em APIs REST e aplicações CRUD, este projeto teve como objetivo explorar outro tipo de problema: a construção de um pipeline ETL. Em vez de trabalhar com o modelo tradicional de requisição e resposta, o foco foi automatizar a extração, transformação e organização de dados em lote, além de ampliar minha experiência com Python e processamento de dados.

Links

Repositório: <https://github.com/Vitormjere/football-etl-pipeline>

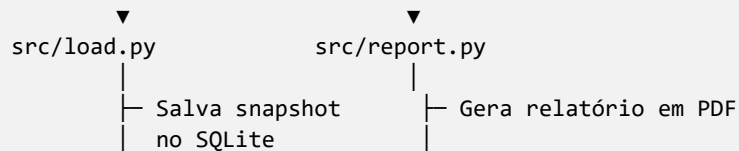
Stack utilizada

Camada	Tecnologias
Extract	Python 3.13, requests, python-dotenv
Transform	Funções puras da biblioteca padrão (sem dependências externas)
Load	SQLite (módulo sqlite3, nativo, sem ORM)
Report	fpdf2, com fonte Unicode (Arial TTF)
Testes	pytest (unitários, com parametrização)
Automação	Windows Task Scheduler
API externa	football-data.org (free tier)

2. Arquitetura Geral

O projeto foi organizado seguindo o fluxo de um pipeline ETL. Cada etapa (extração, transformação e carga) possui seu próprio módulo dentro da pasta `src/`, enquanto o `main.py` é responsável por executar todo o processo na ordem correta





Procurei manter toda a lógica de transformação dentro do transform.py. Esse módulo não faz requisições para a API nem acessa o banco de dados; ele apenas recebe os dados, processa as informações e devolve o resultado. Isso deixou a lógica bem mais fácil de testar, já que os testes não dependem de internet, token de acesso ou de um banco de dados funcionando

```
extract.py -> depende apenas de requests e dotenv (nada do projeto)
transform.py -> não depende de extract nem de load (funções puras)
load.py -> depende apenas dos dados já transformados
report.py -> depende apenas dos dados já transformados
main.py -> depende de todos; orquestra o fluxo completo
```

3. Extract - Coleta de Dados

3.1 Decisões técnicas e por quê

Escolha da API: football-data.org

Escolha: Utilizei a API football-data.org para obter os dados das competições.

Por que escolhi essa API: Ela oferece todas as competições que o projeto precisava, como Premier League, Champions League e La Liga, além de um limite de 10 requisições por minuto no plano gratuito. Como o pipeline é executado periodicamente e não precisa de dados em tempo real, esse limite atendeu perfeitamente às necessidades do projeto.

Alternativa considerada: Também avaliei a API-Football, disponível pela RapidAPI. Apesar de oferecer informações mais detalhadas, principalmente sobre jogadores, o plano gratuito possui um limite diário de requisições bem menor, o que seria mais restritivo para um pipeline que consulta várias competições

Autenticação via header HTTP

Escolha: O token da API é armazenado em uma variável de ambiente (.env) e enviado no cabeçalho X-Auth-Token em cada requisição.

Por que escolhi essa abordagem: Prefiro não deixar a chave da API diretamente no código, evitando expor informações sensíveis no repositório. Para isso, utilizei a biblioteca python-dotenv, que carrega automaticamente as variáveis definidas no arquivo .env

Funções parametrizadas por temporada

Escolha: As funções get_standings e get_matches recebem a temporada como parâmetro opcional.

Por que escolhi essa abordagem: Enquanto desenvolvia o projeto, a temporada 2026/27 ainda não havia começado e, por isso, as tabelas estavam vazias. Para conseguir testar a lógica de transformação com dados reais, utilizei os dados da temporada de 2025. Tornar a temporada um parâmetro permitiu reutilizar as mesmas funções para diferentes anos, sem precisar duplicar código

3.2 Exemplo de código

```
def get_matches(competition_code, status=None, season=None):
    url = f"{BASE_URL}/competitions/{competition_code}/matches"
    headers = {"X-Auth-Token": API_KEY}
    params = {}

    if status:
        params["status"] = status
    if season:
        params["season"] = season

    response = requests.get(url, headers=headers, params=params)

    return response.json()
```

4. Transform - Processamento e Regras

4.1 Design: funções puras, sem I/O

Isolamento total de efeitos colaterais

Escolha: Mantive o transform.py responsável apenas pela transformação dos dados. Ele não faz chamadas para a API nem acessa o banco de dados; apenas recebe os dados já extraídos, aplica as regras de negócio e retorna o resultado.

Por que escolhi essa abordagem: Separar a lógica de transformação dos demais componentes tornou o código mais organizado e muito mais fácil de testar. Como essas funções não dependem de internet, token de acesso ou banco de dados, consigo validá-las rapidamente utilizando apenas o pytest e dados simulados

4.2 Cálculo de forma recente e aproveitamento

Para calcular a forma recente de um time, a função `get_team_form` primeiro seleciona todas as partidas em que ele participou e as ordena por data, garantindo que o processamento seja feito em ordem cronológica, mesmo que a API retorne os jogos em outra sequência. Em seguida, ela analisa as cinco partidas mais recentes e converte cada resultado em **V** (vitória), **E** (empate) ou **D** (derrota), sempre considerando o ponto de vista do próprio time.

A função `calcular_aproveitamento` utiliza essa sequência para calcular o percentual de pontos conquistados em relação ao máximo possível nas últimas cinco partidas. Assim, um time com quatro vitórias e um empate terá um aproveitamento maior do que outro com três vitórias e duas derrotas, por exemplo

```
def get_match_result(match, team_id):
    winner = match["score"]["winner"]
    is_home = match["homeTeam"]["id"] == team_id

    if winner == "DRAW":
        return "E"
    elif winner == "HOME_TEAM" and is_home:
        return "V"
    elif winner == "AWAY_TEAM" and not is_home:
        return "V"
    else:
        return "D"
```

4.3 Tratamento de caso extremo: times sem partidas anteriores

Guard clause para listas de forma vazias

Escolha: A função `calcular_aproveitamento` retorna 0.0 quando a lista de partidas está vazia, antes de realizar qualquer cálculo.

Por que escolhi essa abordagem: Durante os testes encontrei um problema com alguns times recém-promovidos, como Sunderland, Leeds e Ipswich. Como eles não haviam disputado a temporada anterior na mesma competição, não existiam partidas para analisar e a divisão acabava gerando um `ZeroDivisionError`. Além de resolver esse caso, esse tratamento também faz sentido para o início de qualquer temporada, quando ainda há poucos ou nenhum jogo disputado. Assim, a função continua funcionando corretamente mesmo quando não há dados disponíveis

5. Load - Persistência do Histórico

5.1 Decisões técnicas e por quê

SQLite em vez de um banco com servidor

Escolha: Utilizei o SQLite para armazenar o histórico do pipeline, mantendo todos os dados em um único arquivo (`pipeline.db`).

Por que escolhi essa abordagem: Para este projeto, não fazia sentido utilizar um banco de dados com servidor. O SQLite é leve, não precisa de instalação ou configuração adicional e atende perfeitamente à necessidade de armazenar o histórico das execuções. Além disso, foi uma oportunidade de trabalhar com uma solução diferente da que utilizei nos projetos em C#, mostrando que a escolha da tecnologia depende do problema que está sendo resolvido, e não apenas da ferramenta com a qual já estou acostumado

SQL puro via `sqlite3`, sem ORM

Escolha: Todo o acesso ao banco foi feito utilizando o módulo `sqlite3`, com comandos SQL escritos manualmente, sem o uso de um ORM.

Por que escolhi essa abordagem: Nos meus projetos em C# já utilizei bastante o Entity Framework Core, então aproveitei este projeto para trabalhar em um nível mais próximo do banco de dados. Escrever as consultas manualmente me ajudou a praticar SQL e a entender melhor o que acontece por trás de um ORM. Todas as consultas utilizam parâmetros (?) em vez de concatenação de strings, evitando problemas como SQL Injection

Modelagem por snapshot, não normalizada

Escolha: Cada execução do pipeline grava uma nova "fotografia" dos dados em uma única tabela (`team_snapshots`), registrando o estado de cada time naquele momento.

Por que escolhi essa abordagem: Como o objetivo era acompanhar a evolução dos times ao longo do tempo, um modelo baseado em snapshots tornou as consultas muito mais simples. Para comparar o desempenho de uma equipe entre diferentes execuções, basta filtrar pelo nome do time e ordenar os registros pela data, sem precisar fazer junções entre várias tabelas

6. Report - Geração do PDF

6.1 Decisões técnicas e por quê

PDF em vez de Excel

Escolha: O relatório final é gerado em PDF utilizando a biblioteca `fpdf2`.

Por que escolhi essa abordagem: A ideia era gerar um relatório simples e fácil de consultar, semelhante a um boletim com o resumo das competições. Nesse caso, um PDF faz mais sentido do que uma planilha, já que o objetivo é apresentar as informações, e não permitir que elas sejam editadas. Além disso, o arquivo pode ser compartilhado ou incluído no portfólio com facilidade

6.2 Problema real: caracteres especiais em nomes de times

Problema: Durante os testes, percebi que alguns nomes de times faziam a geração do PDF falhar. A fonte padrão do fpdf2 (Helvetica) não oferece suporte completo a caracteres Unicode, então nomes contendo caracteres especiais, como `ğ`, geravam uma exceção `UnicodeEncodeError`.

Solução: Para resolver esse problema, registrei uma fonte TrueType com suporte a Unicode utilizando o método `pdf.add_font()`. Como o desenvolvimento foi feito no Windows, utilizei a fonte Arial disponível no próprio sistema operacional. Assim, o relatório passou a suportar corretamente nomes de equipes de diferentes países, sem interromper a geração do PDF

7. Automação — Execução Agendada

7.1 Decisões técnicas e por quê

Windows Task Scheduler em vez de GitHub Actions

Escolha: O pipeline é executado automaticamente pelo **Agendador de Tarefas do Windows**, que o inicia semanalmente sem necessidade de intervenção manual.

Por que escolhi essa abordagem: Como meus projetos anteriores já utilizavam o GitHub Actions para automação e deploy, aproveitei este projeto para explorar outra forma de automação, desta vez no próprio sistema operacional. Além disso, como o histórico é armazenado em um banco SQLite local, executar o pipeline na mesma máquina facilita a persistência dos dados entre as execuções, sem precisar configurar um armazenamento externo.

Alternativa considerada: Também considerei utilizar o GitHub Actions com execução agendada (`schedule: cron`). Essa abordagem tem a vantagem de rodar na nuvem, sem depender do computador estar ligado. Porém, como cada execução acontece em um ambiente temporário, seria necessário utilizar um banco de dados ou outro tipo de armazenamento externo para preservar o histórico do SQLite entre as execuções, o que aumentaria a complexidade do projeto

7.2 Configuração da tarefa

A tarefa aponta diretamente para o interpretador Python dentro do ambiente virtual do projeto (não o Python global), com o argumento `main.py` e o diretório inicial configurado explicitamente para a raiz do projeto, necessário porque o Agendador de Tarefas não herda o contexto de um terminal ativo (sem `venv` ativado, sem diretório atual definido).

```
Programa/script: C:\Projetos\football-etl-pipeline\venv\Scripts\python.exe
Argumentos:      main.py
Iniciar em:      C:\Projetos\football-etl-pipeline
Disparador:      Semanalmente, aos domingos as 13:00
```

8. Testes Automatizados

A suíte de testes cobre a lógica do módulo `transform.py`, a única parte do pipeline livre de efeitos colaterais e, portanto, a mais crítica de testar isoladamente. Ao todo, 10 testes cobrem três funções:

- `get_match_result`: 4 cenários (vitória do mandante, derrota do mandante, vitória do visitante, empate)
- `calcular_aproveitamento`: 5 casos parametrizados (lista vazia, todas vitórias, todas derrotas, mix simples, mix com arredondamento)
- `get_team_form`: ordenação cronológica correta mesmo com partidas fora de ordem na lista de entrada

8.1 Testes parametrizados

Em vez de repetir uma função de teste para cada cenário de aproveitamento, foi usado o decorator `@pytest.mark.parametrize`, que roda a mesma função de teste várias vezes com entradas e saídas diferentes, equivalente ao uso de `[Theory]` + `[InlineData]` no xUnit.

```
@pytest.mark.parametrize("forma, esperado", [
    ([], 0.0),
    (["V", "V", "V"], 100.0),
    (["D", "D", "D"], 0.0),
    (["V", "D"], 50.0),
    (["V", "E", "D"], 44.4),
])
def test_calcular_aproveitamento(forma, esperado):
    resultado = calcular_aproveitamento(forma)
    assert resultado == esperado
```

8.2 Resolução de import: conftest.py

Problema: o pytest, por padrão, não reconhecia a pasta `src/` como um pacote importável a partir da pasta `tests/`, resultando em `ModuleNotFoundError: No module named 'src'` ao rodar a suíte.

Solução: criação de um arquivo `conftest.py` vazio na raiz do projeto, sua simples presença sinaliza ao pytest qual diretório usar como referência de raiz para resolução de imports.

9. Desafios Reais e Como Foram Resolvidos

Esta seção documenta problemas reais enfrentados durante o desenvolvimento, úteis para revisão antes de entrevistas técnicas, pois mostram raciocínio de depuração aplicado a um projeto real, não apenas teoria.

Comando 'python' não reconhecido no Windows

Problema: o comando `python` no PowerShell abria um atalho da Microsoft Store em vez de executar o interpretador, mesmo com o Python já instalado.

Solução: identificado que o Python estava instalado e acessível via `py` (o launcher oficial do Windows), mas o alias automático de `'python'` para a Store não havia sido desabilitado. A solução prática foi usar `py` como comando padrão em todo o fluxo de trabalho.

PowerShell bloqueando a ativação do ambiente virtual

Problema: o comando `venv\Scripts\Activate.ps1` era bloqueado pela política de execução de scripts padrão do PowerShell.

Solução: liberação pontual da política para o processo atual (`Set-ExecutionPolicy -Scope Process - ExecutionPolicy RemoteSigned`), sem alterar a política global do sistema.

ModuleNotFoundError: No module named 'requests'

Problema: ao abrir uma nova sessão de terminal, o comando `py main.py` falhava porque o ambiente virtual daquela sessão específica não estava ativado, as bibliotecas instaladas só existem dentro do `venv` ativo.

Solução: reativação do ambiente virtual na nova sessão antes de rodar qualquer comando; reforçou a importância de confirmar o prefixo (`venv`) no terminal antes de qualquer execução.

ModuleNotFoundError: No module named 'extract'

Problema: `transform.py` importava `extract` diretamente (`from extract import ...`), o que funcionava ao rodar o arquivo isoladamente de dentro de `src/`, mas quebrava ao ser importado por `main.py`, que trata `src/` como um pacote.

Solução: padronização do import como `from src.extract import ...`, consistente com a perspectiva da raiz do projeto, executando os módulos individuais com `py -m src.transform (flag -m)` em vez de chamar o arquivo diretamente.

ZeroDivisionError ao calcular aproveitamento

Problema: `get_standings` (sem parâmetro de temporada) retornava a tabela da temporada atual, ainda não iniciada, enquanto `get_matches` buscava partidas da temporada anterior, um descompasso que produzia times sem nenhuma partida correspondente (recém-promovidos) e uma divisão por zero no cálculo de aproveitamento.

Solução: alinhamento do parâmetro `season` entre as duas chamadas, garantindo que `standings` e `matches` sempre se referissem à mesma temporada, combinado com a `guard clause` de lista vazia descrita na seção 4.3.

UnicodeEncodeError na geração do PDF

Problema: a fonte padrão do `fpdf2` (Helvetica) não suportava um caractere especial em um nome de time, interrompendo a geração do relatório com uma exceção.

Solução: troca para uma fonte TTF com suporte Unicode completo (Arial), registrada explicitamente a partir do sistema operacional, detalhada na seção 6.2.

PDF parecendo não atualizar entre execuções

Problema: após corrigir o código e gerar o relatório novamente, o conteúdo visual parecia não ter mudado.

Solução: identificado que a causa raiz era um bloco de código (o laço que desenha as tabelas por liga) que não havia sido salvo corretamente no arquivo `report.py` entre uma edição e outra, confirmado ao imprimir a quantidade de times antes de gerar o relatório, isolando o problema entre 'dados incorretos' e 'renderização incorreta'.

URL de exemplo não substituída no git remote

Problema: o comando `git remote add origin` foi executado com o placeholder literal (`SEU_USUARIO/NOME_DO_REPO`) em vez da URL real do repositório, resultando em 'repository not found' ao tentar o push.

Solução: remoção do remote incorreto (`git remote remove origin`) e nova configuração com a URL correta do repositório no GitHub.

Agendador de Tarefas recusando salvar a credencial

Problema: a opção 'Executar estando o usuário conectado ou não' exige a senha da conta do Windows para ser salva, mas o login era feito via PIN, o Agendador não reconhecia esse tipo de credencial, apresentando erro de conta ou senha inválida.

Solução: troca para a opção 'Executar somente quando o usuário estiver conectado', que não exige senha salva, trade-off aceitável para um projeto de portfólio: a tarefa passa a depender de uma sessão ativa no Windows no horário agendado.

10. Como Rodar o Projeto Localmente

10.1 Configuração

```
git clone https://github.com/Vitormjere/football-etl-pipeline.git
cd football-etl-pipeline
py -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Criar um arquivo `.env` na raiz do projeto, com uma chave gratuita obtida em football-data.org:

```
FOOTBALL_DATA_API_KEY=seu_token_aqui
```

10.2 Execução

```
py main.py
```

Busca os dados mais recentes das três competições, salva um snapshot em pipeline.db e gera relatorio.pdf na raiz do projeto.

10.3 Testes

```
pytest -v
```

Requer Python 3.13+ e uma chave de API válida do football-data.org.